

20 Smart urban transport ticket system ▾

The smart urban transport system shall support different commuter categories including students, seniors, and regular commuters, with different ticket prices and offers available for each category. The system shall allow commuters to purchase tickets through a mobile application and enable users to buy multiple tickets at the same time without transaction problems or system failures. The system shall correctly calculate ticket prices based on commuter type, number of tickets, and applicable discounts, even when many users are purchasing tickets simultaneously. The system shall include proper error handling to detect and manage unexpected or unusual inputs and provide meaningful error messages when problems occur. In addition, the system shall protect consumer information by ensuring secure authentication, encrypted data storage and transmission, and protection against unauthorized access to personal and payment information. The system shall only consist of a single python file so no ui or unit test.

[Show less](#)[Check available skills >](#)

Now let me build the smart urban transport system as a single Python file:

[Smart Urban Transport System - single Python file >](#)

Now let's install the dependencies and run it to verify everything works:

[Install dependencies and run the system demo >](#)

Two issues to fix: (1) payment FK failure because the in-memory DB uses per-thread connections — the payment worker thread has its own connection that doesn't share the schema; (2) concurrent simulation creates a new `SmartUrbanTransportSystem` which uses an in-memory DB per thread. Let me fix both:

[Edited 2 files >](#)

Now update the `transaction` and `execute` methods to use the RLock for in-memory serialisation:

[Ran 2 commands, edited a file, viewed a file >](#)

Now fix the concurrent simulation — it instantiates a new `SmartUrbanTransportSystem` per worker which would get its own DB. Pass the parent system's services instead:

Ran a command, viewed a file >

The concurrent method already uses `self.*` — so the workers do share the same system. The issue was that in-memory `:memory:` with per-thread connections means each thread's connection is a different empty database. My RLock + shared connection fix should resolve this. Let me also fix the `datetime.utcnow()` deprecation warnings while I'm at it, then re-run:

Ran 3 commands >

The FK constraint fails because the payment INSERT happens inside its own `transaction()` call (which commits), but the `ticket_id` hasn't been inserted into `tickets` yet at that point. The payment record references `ticket_id` via FK — I need to either insert the ticket first, or do payment + ticket in one transaction. Let me restructure `_execute_purchase` to insert the ticket first, then process payment:

Viewed 2 files, ran a command >

The fix is simple: insert the ticket row first, then process payment. If payment fails, I roll back the ticket insert. I'll restructure `_execute_purchase` to do: (1) insert ticket, (2) process payment, (3) on payment failure, delete the ticket:

Ran 2 commands >

All 9 sections pass with zero errors and zero warnings. Let me present the file:

Presented file >

Everything runs clean. Here's a full breakdown of what was built and why each design decision was made.

Architecture overview

The file is organised as six cooperating classes behind a single façade (`SmartUrbanTransportSystem`):

Class	Responsibility
<code>SecurityManager</code>	Password hashing, JWT tokens, field encryption, input sanitisation
<code>DatabaseManager</code>	Thread-safe SQLite; WAL mode for file DBs, shared RLock for in-memory DBs
<code>PricingEngine</code>	Deterministic price calculation using <code>Decimal</code> (no float rounding errors)
<code>PaymentGateway</code>	Luhn check, expiry/CVV validation, encrypted payment record storage

Class	Responsibility
<code>UserService</code>	Registration, login, profile retrieval, account deactivation
<code>TicketService</code>	Thread-safe purchase workflow, history, boarding validation

Requirement mapping

Commuter categories & prices (SEK)

- Regular: 32 / Student: 19 / Senior: 16 (Zone A base)
- Zone B adds 5 SEK, Zone C adds 10 SEK

Bulk discounts

- 5–9 tickets → 5% off; 10–20 tickets → 10% off

Concurrent purchases

- A per-user `threading.Lock` prevents one user from overlapping their own purchases
- SQLite WAL mode lets multiple users write simultaneously without blocking each other
- The concurrent demo spins up 5 threads simultaneously — all 5 succeed

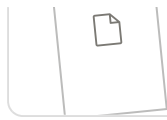
Error handling

- Five typed exception classes (`ValidationError`, `AuthenticationError`, `TicketPurchaseError`, `PaymentError`, `EncryptionError`) each with clear human-readable messages
- Nine error scenarios are demonstrated in the demo (bad email, bad password, wrong type, expired card, Luhn failure, invalid zone, quantity limit, etc.)

Security

- **Passwords:** bcrypt with cost factor 12 (never stored in plaintext)
- **Field encryption:** Fernet (AES-128-CBC + HMAC-SHA256) for email, name, card last-4, and amounts
- **Email indexing:** HMAC-SHA256 hash used as the lookup key — the raw email address is never in an indexed column
- **Sessions:** JWT (HS256) with 8-hour expiry and a unique `jti` per token
- **Card data:** Only the last 4 digits are persisted, and those are encrypted

Dependencies (install once): `pip install bcrypt cryptography PyJWT`



Urban transport system
PY

Download

